Universidad de los Andes > FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Software Architecture

Assignment 1 Brief

Grupo: 6

Integrantes:

Domingo Fuenzalida Izquierdo

Martin Hargous Fuentes

Javier Valdera Contador

Marcelo Rojas Rozas

Profesora:

Jose Luis Assadi

19 de Agosto del 2026

1. Language, framework and database engine

Component	Version / detail
Language	JavaScript (Node.js; node:20-alpine in the Dockerfile)
Web framework	Express 5.2.1
DB engine	SQLite 3 (driver sqlite3 6.0.1), file data/database.sqlite
Abstraction layer	Sequelize 6.37.8 (ORM)
Password hashing	bcrypt 6.0.0, 10 rounds
Frontend	No framework: HTML with JavaScript
Container	Dockerfile + docker-compose, port 3000
Build step	None; the frontend is served statically from public/

2. The group's familiarity with these technologies

- **JavaScript (Express):** There was previous experience in previous university projects. The assigned framework matched what we already knew how to use.
- **SQLite**: First time for all. Nobody had used it before, but it resembled PostgreSQL in several ways, so the learning curve was not steep.
- **Frontend tools:** Fairly familiar. All group members had used a lot of times plain HTML and JavaScript for different web projects
- **Sequelize:** No experience at all.

3. How the framework connects to the database

There are three layers with separate roles — Express handles HTTP requests and routing, SQLite is the database where the data lives, and Sequelize is the bridge that lets the application talk to the database using JavaScript instead of raw SQL.

- **Instantiation "config/database.js":**
  - A "new Sequelize({ dialect: 'sqlite', storage: <path>, logging: false })" object is created.
  - The path points to "data/database.sqlite", resolved with path.join.
- **Sequelize as an ORM (Object-Relational Mapper):**
  - Maps JavaScript classes to tables and instances to rows.
  - Each model is declared with "sequelize.define(name, attributes, options)" in the models folder, specifying type (DataTypes.STRING, TEXT, INTEGER, DATEONLY), nullability, uniqueness and validations.
  - It translates calls such as findAll, findByPk, create, update, destroy into parameterized SQL queries
- **Associations**: "models/index.js" declares the four relationships:
  - Author.hasMany(Book) → Book.belongsTo(Author)
  - Book.hasMany(Review) → Review.belongsTo(Book)
  - User.hasMany(Review) → Review.belongsTo(User)
  - Book.hasMany(SaleByYear) → SaleByYear.belongsTo(Book)

  All of them with "onDelete: 'CASCADE'". The "include" option in queries uses these associations to generate the JOINs automatically.

- **Schema:** No migrations are used, so "index.js" calls "sequelize.sync({ alter: true })" on every startup.
- **Foreign keys in SQLite:** SQLite does not enforce them by default. database.js registers an afterConnect hook that runs "PRAGMA foreign_keys = ON;" on every new connection.
- **Model hooks:** "models/User.js" uses "beforeCreate" and "beforeUpdate" to hash the password with "bcrypt" before the INSERT/UPDATE, so that the logic lives in the model and not in the routes. The seed calls

"bulkCreate(..., { individualHooks: true })" precisely so that the hooks starts during the bulk load.

- **Validations:** declared in the model and executed in JS before using the database (score between 1 and 5, sales ≥ 0, isEmail on the address). The composite unique constraint (book_id, year) on SaleByYear does get pushed down to SQL as a unique index.

4. Time and difficulties per stage

While the introduction of certain unfamiliar tools required a standard period of adjustment, all stages of development were completed without major impediments, as the team already possessed foundational experience with the majority of the assigned stack.

4.1 Time distribution per stage

Stage	Time dedicated
Database Design	2 hours
Database Setup	3 hours
Backend API	9 hours
Frontend Views	4 hours
ReadME and Brief	2 hours

5. Time lost or gained because of the assigned framework

- **Frameworks that helped gain time:**
  - **Express:** As a previously known technology, the framework assignment did not force the team to invest time in learning a new system for routing and HTTP requests.
  - **Frontend tools (HTML and JavaScript):** Extensive prior experience across various projects meant no time was lost to familiarization, allowing for immediate implementation of the user interface.
- **Frameworks that helped lose time:**
  - **SQLite:** Its strong resemblance to PostgreSQL cushioned the learning curve. The time spent adapting was low and disproportionately small considering it was the team's first experience with the engine.

  - **Sequelize:** With no prior experience within the group, learning how to configure its ORM abstraction, define models, and handle associations constituted the primary learning overhead during the development process, and the most part of the time invested on the backend development and database setup was learning and adapting to use it.